

Número de Orden:
Nombre y Apellido:

Libreta Universitaria:
Cantidad de Hojas Entregadas: 3

Ingeniería del Software II

Parcial #1 - Generación Automática de Tests

Ej1	Ej2	Ej3	Ej4	Nota	Aprobado/Desaprobado
B	B	M+	B	8	A

El examen es a libro abierto. Agregar número de orden, nombre y apellido y LU a cada hoja. Cada ejercicio se evaluará como **Bien**, **Regular** o **Mal**. Para aprobar el examen es necesario tener al menos 2 ejercicios Bien y al menos 1 ejercicio Regular.

Bien = 2,5p, Regular = 1,25p, Mal = 0p.

Ejercicio 1

Sea el siguiente programa:

```

1 def test_me(n: int, k: int) -> int:
2     r: int = 0
3     i: int = 0
4     k = k % 3
5     while i < k: # Condition C1
6         if k % 2 == 0: # Condition C2
7             r = r + n
8         else:
9             r = r - n
10            i = i + 1
11    return r

```

Exhibir usando el operador de mutación Relational Operator Replacement (ROR)¹:

- Un mutante con un test que lo detecte. Indicar cuál es el test case que lo detecta.
- Un mutante equivalente tal que no exista ningún test case que lo detecte.

En ambos casos indicar la línea que se modificó.

Ejercicio 2

Sea el programa `test_me` del ejercicio #1.

- Completar la siguiente tabla con la ejecución simbólica dinámica del programa de forma manual, indicando para cada iteración:

- El input concreto utilizado
- La condición de ruta (i.e. "path condition") que se produce al ejecutar el input concreto, asumiendo que el valor simbólico inicial es $n = n_0$ y $k = k_0$.
- La fórmula lógica (no es necesario escribir-

la en SMTLib) que se envía al demostrador de teoremas de acuerdo al algoritmo de ejecución simbólica dinámica.

- El resultado posible que podría producir un demostrador de teoremas (ej: Z3).

- Describir el árbol de cómputo del programa explorado durante la ejecución simbólica dinámica del programa

Iteración	Input Concreto	Condición de Ruta	Fórmula enviada al demostrador	Resultado posible
1	$n=0, k=0$	...	...	...
2	...	...	...	...
...	...	...	...	...

¹Remplaza una comparación aritmética por $>, <, \geq, \leq, ==, !=$

Ejercicio 3

Sea el programa `test_me` del ejercicio #1 y el siguiente test suite:

```
def test_0(self):
    self.assertEqual(0, test_me(0,7))
def test_1(self):
    self.assertEqual(0, test_me(0,13))
```

- Dibujar el grafo de control de flujo.
- Dibujar el grafo de control de dependencia (recordar marcar los arcos *true* y *false* que correspondan).
- Una vez ejecutado el test suite:
 - ¿Cuál es el valor del approach level para cada nodo del grafo de control de dependencia? Justificar.
 - Suponer que $K = 5$, ¿cuál es el valor de la distancia de branch no normalizada para cada decisión? Justificar.
 - ¿Cuál es el cubrimiento de branches que logra el test suite?

Ejercicio 4

Sea el siguiente programa:

```
1 def parse_config(config: str) -> Dict[str, str]:
2     lines: list[str] = config.splitlines()
3     parsed: Dict[str, str] = {}
4     i: int = 0
5     while i < len(lines):
6         line: str = lines[i].strip()
7         if line == "" or line.startswith("#"):
8             i += 1
9             continue
10        if "=" in line:
11            parts: list[str] = line.split("=", 1)
12            key: str = parts[0].strip()
13            value: str = parts[1].strip()
14            parsed[key] = value
15        i += 1
16    return parsed
```

Asumiendo que tenemos un **boosted greybox fuzzer** con exponente $a = 5$, y el siguiente conjunto inicial de inputs:

```
#1 "username = alice\npassword = secret\n"
#2 "  host = localhost\n  port = 8080\n"
#3 ""
#4 "#Db config\n\ndb_name = test_db\n\n\ndb_user = admin \n\ndb_pass = 1234\n"
#5 "#Db config\n\ndb_name = test_db #db name \n\ndb_pass = 1234\n"
#6 "theme = dark\n"
#7 "timeout=30\n"
#8 "language = Python\nversion = 3.11\n"
#9 "user = bob\nemail = bob@example.com\nactive = true\n"
#10 "server = myserver\nprotocol = https\nport = 443\n"
```

- Indicar el camino de ejecución de cada input (qué condicionales se toman y si lanza excepción o no). Recordar que un camino es una lista y no un conjunto de instrucciones.
- Indicar la energía de cada input luego de la ejecución del conjunto inicial.
- ¿Cuál es la probabilidad que el fuzzer elija el input #10 para mutar?

- 1) a) Consideremos el mutante que resulta de aplicar el operador ROR sobre la guard "K % 2" en la línea 6 y modificarlo por "K % 2 != 0".

"K % 2 == 0"

Este mutante es distinguible por el siguiente test:

def test_mutant():

— assertEqual(-100, test_me(100, 1))

Ya que a la mutación realizada tiene el efecto de verificar imparidad donde originalmente se verificaba paridad, logrando que ^{con} K = 1 pase de ejecutarse la línea 9 a ejecutarse la 7, lo cual resulta en un resultado de 100 en vez de -100.

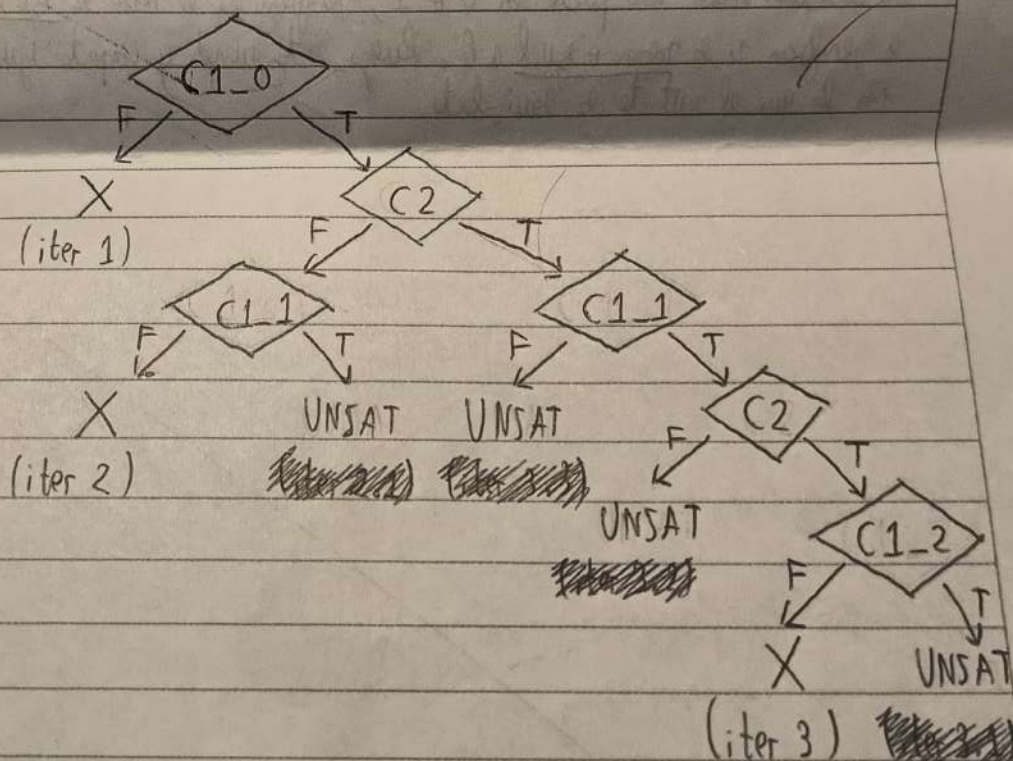
- b) Consideremos el mutante que resulta de aplicar ROR sobre la guard "K % 2 == 0" en la línea 6 y modificarlo por "K % 2 <= 0". Como el resto de la división por 2 de cualquier entero solo puede ser 0 o 1, verifican si el resto es igual a 0 es equivalente a verificar si es menor o igual a 0. Luego, este guard se comporta igual al programa original, por lo que el mutante es equivalente.

2) a)

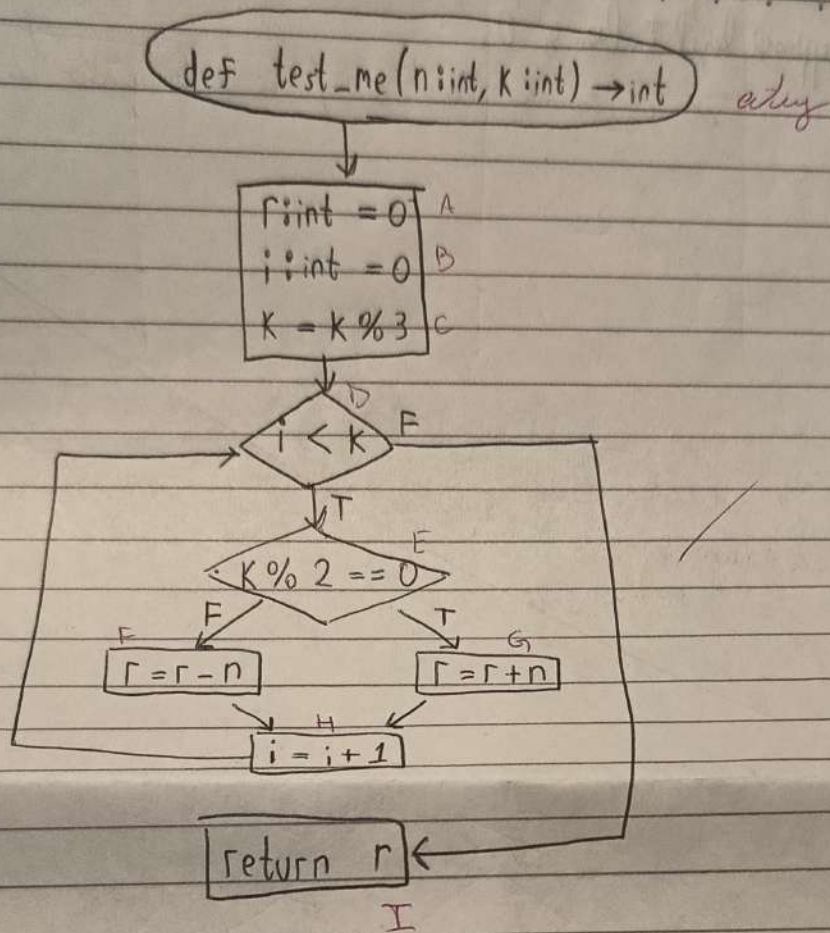
Iter.	Input Concreto	Condición de Ruta	Fórmula al Demostrador	Resultado
1	$n=0, k=0$	$\neg C1_0$	$C1_0$	$n_0=0, k_0=1$
2	$n=0, k=1$	$C1_0 \wedge \neg C2 \wedge \neg C1_1$	$C1_0 \wedge \neg C2 \wedge C1_1$	UNSAT
(2.1)			$C1_0 \wedge C2$	$n_0=0, k_0=2$
3	$n=0, k=2$	$C1_0 \wedge C2 \wedge C1_1 \wedge \neg C1_2$	$C1_0 \wedge C2 \wedge C1_1 \wedge C2 \wedge C1_2$	UNSAT
(3.1)			$C1_0 \wedge C2 \wedge C1_1 \wedge \neg C2$	UNSAT
(3.2)			$C1_0 \wedge C2 \wedge \neg C1_1$	UNSAT
(3.3)			END	END

Utilice las siguientes abreviaturas: $C1_i \equiv "i < (k_0 \% 3)"$
 $C2 \equiv "(k_0 \% 3) \% 2 == 0"$

b) Usando las mismas fórmulas $C1_i$ y $C2$, el árbol de cómputo explorado es:

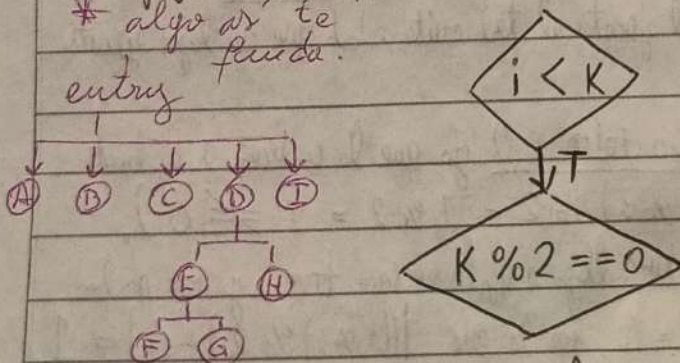


3 a



b) El grafo de control dependiente queda:

* algo así te queda.



* ojo, así no se arma el grafo de control dependiente

Demuestra que entre las dos condiciones del programa, se necesitan hacer verdadero C1 para poder alcanzar (y hacer verdadero o falso) C2.

c) i) El approach level de C1 es 0, ya que no depende de ningún otro condicional y fue evaluado. En cambio, ~~para~~ C2 depende de C1, pero en la ejecución del test suite llegamos a evaluarlo, así que su approach level también es 0.

el approach level es false cada nodo de DG.

ii) Para la decisión " $i < K$ " de la línea 5, las distancias true y false son ambas 0, ya que la condición es evaluada tanto a true como a false al ejecutar el test suite, lo que se logra ejecutando el ciclo al menos una vez.

Para " $K \% 2 == 0$ " en la línea 6, la distancia false es 0 ya que la condición es evaluada a false con $K = 7$ (ya que $7 \% 2 = 1 \neq 0$).

La distancia true en cambio es 1, ya que nunca llega a evaluarse como true, y por valores de K en el test suite vale que $K \% 3 == 1$, por lo que $|(K \% 3) \% 2 - 0| = 1$, y este es el valor de la distancia true.

iii) Como vimos en el ítem anterior, el test suite logra cubrir 3 de las 4 decisiones del programa. Luego, el cubrimiento de ramas alcanzado es:

$$\frac{\text{branches alcanzados}}{\text{branches totales}} = \frac{3}{4} = \boxed{75\%}$$

4 Utiliza la siguiente abreviatura de comins:

C1: [2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15,
5, 6, 7, 10, 11, 12, 13, 14, 15, 5, 16]

C2: [2, 3, 4, 5, 16]

C3: [2, 3, 4, 5, 6, 7, 8, 9,
5, 6, 7, 10, 11, 12, 13, 14, 15,
5, 6, 7, 8, 9,
5, 6, 7, 10, 11, 12, 13, 14, 15,
5, 6, 7, 10, 11, 12, 13, 14, 15, 5, 16]

C4: [2, 3, 4, 5, 6, 7, 8, 9,
5, 6, 7, 10, 11, 12, 13, 14, 15,
5, 6, 7, 10, 11, 12, 13, 14, 15, 5, 16]

C5: [2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 5, 16]

C6: [2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15,
5, 6, 7, 10, 11, 12, 13, 14, 15,
5, 6, 7, 10, 11, 12, 13, 14, 15, 5, 16]

En este ejercicio asumí que splitlines() no devuelve una línea vacía al haber un "\n" al final.

Los comins, frecuencias y energías de cada input inicial quedo de acuerdo a la tabla:

Input	Camino	Frecuencia	Energía
# 1	C1	3	$1/3^5$
# 2	C1	3	$1/3^5$
# 3	C2	1	$1/1^5$
# 4	C3	1	$1/1^5$
# 5	C4	1	$1/1^5$
# 6	C5	2	$1/2^5$
# 7	C5	2	$1/2^5$
# 8	C1	3	$1/3^5$
# 9	C6	2	$1/2^5$
# 10	C6	2	$1/2^5$

c) La probabilidad de ir al input #10 es el cociente entre su energía y la suma de las energías de todos los inputs posibles; es decir:

$$\frac{\frac{1}{2^5}}{3 \cdot \frac{1}{3^5} + 1 \cdot \frac{1}{1^5} + 1 \cdot \frac{1}{1^5} + 1 \cdot \frac{1}{1^5} + 2 \cdot \frac{1}{2^5} + 2 \cdot \frac{1}{2^5}}$$

C1 C2 C3 C4 C5 C6